

KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 6 Laboratuvar Föyü

Kalıtım ve Kompozisyon

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Tahmini süre: 2 saat
Gereksinimler: C++20 derleyici (g++/clang++), bir metin editörü

Öğrenme fazları: 1 • Ön-öğretim | 2 • İçerik | 3 • Katılım | 4 • Değerlendirme | 5 • İzleme

1 • Ön-öğretim Neden Önemli?

Kitaplık projesinde üç materyal türünüz olacak: basılı kitap, e-kitap ve sesli kitap. Üçünde de `baslik`, `yazar` ve `yil` *ortaktır*; ama her birinin kendine özgü verisi de vardır (basılıda sayfa sayısı, e-kitapta dosya boyutu, seslide süre ve seslendiren). Üç ayrı bağımsız sınıf yazarsanız bu ortak alanları ve erişimcilerini **üç kez** tekrarlar, birinde bir düzeltme yapınca ötekileri unutursunuz. Bu lab'da ortak olanı *bir kez* bir `Materyal` taban sınıfında toplayacak, farklı olanı türetilmiş sınıflarda tanımlayacak ve sonunda hepsini tek bir `Kutuphane`'nin tek bir döngüde — her materyali kendi türüne göre — yazdırmasını sağlayacaksınız.

1 • Ön-öğretim Ön Bilgi

Bu föy YG I ders notu Bölüm 6 (Kalıtım ve Kompozisyon) üzerine kuruludur ve önceki haftaların şu kavramlarını hazır varsayar: Hafta 3 — sınıf, kapsülleme, `public`/`private` erişim, kurucu/yıkıcı, başlatma listesi ve `this`; Hafta 4 — nesnelerin kurulma/yıkılma sırası, kopya kurucu, `std::unique_ptr`, `std::vector` ve RAII sahipliği; Hafta 5 — bir tipi tam bir değer tipi gibi davrandırma bilinci. `public`/`private`'a bu hafta üçüncü bir erişim düzeyi (`protected`) eklenir. Oluşturacağınız hiyerarşi, Hafta 3'ün `kitaplik` ad uzayına kardeş bir aile olarak girer; `kitaplik::Kitap`'a hiç dokunmazsınız.

Bu föyde adım adım bir sınıf ailesi kuracaksınız: önce ortak veriyi ve davranışı tutan somut bir `Materyal` taban sınıfı, ardından ondan türeyen `BasiliKitap`, `EKitap` ve `SesliKitap` sınıfları, en sonunda da bu materyalleri çokbiçimli olarak yöneten bir `Kutuphane`. Her sınıfı sıfırdan elle yazacak, her adımda derleyip çalıştırarak kalıtımın, kurucu zincirinin, sanal gönderimin ve kompozisyonun nasıl davrandığını gözlemleyeceksiniz.

Bu föyü şöyle kullanın: her adımda önce *Ne yapacağız?* kutusundaki görevi tamamlayın (kodu yazın, ardından verilen kabuk komutunu çalıştırın), sonra kendi sonucunuzu *Beklenen* kutusuyla karşılaştırarak doğru yolda olduğunuzu hemen doğrulayın. Bir şey tutmazsa, sonraki adıma geçmeden o adımı düzeltin — her adım bir öncekinin üstüne inşa edilir. Her birkaç adımda bir gelen *Kontrol Noktası*, o ana kadar yaptıklarınızı bir arada sınamanıza yarar. Adımlar ders notu Bölüm 6'nın dört ünitesiyle sırayla ilerler; her bölümün başında ilgili üniteye bir atıf bulacak, takıldığınızda oraya dönebileceksiniz.

1 Materyal Tabanı ve İlk Türetilmiş Sınıf

(bkz. Ders notu Bölüm 6, Ünite 1 — Neden Kalıtım? is-a İlişkisi, Taban/Türetilmiş Sınıf, `public` Kalıtım, `protected` Üyeler.)

Adım 1: Proje klasörünü kurup `Yazar` ve somut `Materyal` tabanını yazın

Ne yapacağız? `hafta06-projesi` adlı bir klasör oluşturacak, içine `materyal.hpp` (bildirimler) ile `materyal.cpp` (gövdeler) dosyalarını yazacaksınız. `materyal.hpp` içinde `kitaplik` ad uzayında iki sınıf tanımlayın: küçük bir bileşen olan `Yazar` (`ad_`, `ulke_` ve `tamAd()`) ve somut taban `Materyal` (`protected` olarak `baslik_`, `yazar_`, `yil_`; bir kurucu bildirimi; `baslik()` erişimcisi; `virtual void yazdir() const` ve `virtual ~Materyal()` bildirimleri). `materyal.cpp` içine ise `Materyal` kurucusunun, `yazdir()`'inin ve yıkıcısının gövdelerini yazın (her birine bir `std::cout` izleme satırı koyun).

Neden? Her hiyerarşi bir taban sınıfla başlar: ortak veriyi (alanlar) ve ortak davranışı (`yazdir()`) tek yerde toplarsınız — ileride üç türetilmiş sınıf bunları tekrarlamadan devralacak. Alanları `protected` yaparsınız, çünkü `private` olsalardı türetilmiş sınıfların `yazdir()`'i onlara doğrudan erişemezdi; `protected` sınıfa *ve* ondan türeyenlere açıktır, dışarıya ise `private` gibi kapalıdır. `yazdir()` şimdiden `virtual` bildirilir — bu bildirimin "canlanması" (çalışma-zamanı gönderim) Ünite 3'te olacak. `Yazar`'ı ayrı bir sınıf yapmak bir kompozisyon (has-a) örneğidir: bir `Materyal` bir `Yazar` *değildir*, bir `Yazar`'a *sahiptir*.

```
// has-a: Materyal bir Yazar DEGILDIR, bir Yazar'a sahiptir (kompozisyon).
class Yazar {
public:
   Yazar(std::string ad, std::string ulke)
      : ad_(std::move(ad)), ulke_(std::move(ulke)) {}

   const std::string& ad() const { return ad_; }
   const std::string& ulke() const { return ulke_; }
   std::string tamAd() const { return ad_ + " (" + ulke_ + ")"; }

private:
   std::string ad_;
   std::string ulke_;
};
```

```
// Somut taban: yazdir() virtual'dir ama saf sanal DEGILDIR (Materyal m = bk;
// derlenir ve nesne dilimlemesi gozlenebilir kalir).
class Materyal {
public:
   Materyal(std::string baslik, Yazar yazar, int yil);
   const std::string& baslik() const { return baslik_; }
   virtual void yazdir() const;
   virtual ~Materyal();

protected:
   std::string baslik_;
   Yazar yazar_;
   int yil_;
};
```

```
Materyal::Materyal(std::string baslik, Yazar yazar, int yil)
   : baslik_(std::move(baslik)), yazar_(std::move(yazar)), yil_(yil) {
   std::cout << "Materyal kuruldu\n";
}
```

Adım 1 (devamı)

```cpp
void Materyal::yazdir() const {
    std::cout << "Materyal: \"" << baslik_ << "\" - " << yazar_.tamAd()
              << " (" << yil_ << ")\n";
}

Materyal::~Materyal() {
    std::cout << "Materyal yikildi\n";
}
```

`materyal.hpp`'nin başına `#pragma once` ve `#include <memory>`, `#include <string>`, `#include <vector>` satırlarını; tüm bildirimleri `namespace kitaplik { ... }` bloğunun içine yazın. `materyal.cpp`'nin başında `#include "materyal.hpp"` ve `#include <iostream>` bulunmalıdır.

```
>_ Terminal

$ mkdir hafta06-projesi && cd hafta06-projesi
$ g++ -std=c++20 -c materyal.cpp -o materyal.o
```

Beklenen sonuç: Komut hatasız/uyarısız tamamlanır ve dizinde bir `materyal.o` nesne dosyası oluşur. Bu noktada `Materyal` verisini tutabilen, kurulabilen ve kendini yazdırabilen somut bir tiptir; ama henüz ondan türeyen bir sınıf ve çalıştırılabilir bir program yoktur. `-c` yalnız derleme (bağlama yok) yaptığı için `main` olmaması sorun değildir.

Adım 2: `BasiliKitap`'ı `Materyal`'den türetin

Ne yapacağız? `materyal.hpp`'ye `class BasiliKitap : public Materyal` bildirimini (ek `private` alan `sayfaSayisi_`; bir kurucu; `void yazdir() const override;` ve `~BasiliKitap() override;`) ekleyeceksiniz. `materyal.cpp`'ye kurucunun gövdesini (başlatma listesinde *önce* taban `Materyal(...)`'i çağırın) ve `yazdir()`'in override gövdesini yazın; yıkıcının gövdesini de taban yıkıcısındaki kalıba bakarak kendiniz yazın (tek satırlık bir `std::cout << "BasiliKitap yikildi\n";`). Son olarak `kalitim_demo.cpp` adlı bir `main` dosyası oluşturup ilk demoyu yazın.

Neden? `class BasiliKitap : public Materyal`, **public kalıtım**la is-a ilişkisini kurar: bir `BasiliKitap` *bir* `Materyal`'dir ve tabanın tüm `public` arayüzünü (`baslik()`) ile `protected` alanlarını devralır. Kurucunun başlatma listesinde `: Materyal(...)` yazmak **kurucu zincir**idir; taban kurucusunu siz çağırmazsanız derleyici tabanın *varsayılan* kurucusunu arar ve bulamazsa hata verir. `override` anahtar sözcüğü, `yazdir()`'in tabandaki sanal fonksiyonu gerçekten geçersiz kıldığını derleyiciye doğrulatır — imza uyuşmazsa sessizce yeni bir fonksiyon oluşturmak yerine hata alırsınız.

```cpp
class BasiliKitap : public Materyal {
public:
    BasiliKitap(std::string baslik, Yazar yazar, int yil, int sayfaSayisi);
    void yazdir() const override;
    ~BasiliKitap() override;

private:
    int sayfaSayisi_;
};
```

```cpp
BasiliKitap::BasiliKitap(std::string baslik, Yazar yazar, int yil,
                         int sayfaSayisi)
```

Adım 2 (devamı)

```
    : Materyal(std::move(baslik), std::move(yazar), yil),
      sayfaSayisi_(sayfaSayisi) {
    std::cout << "BasiliKitap kuruldu\n";
}

void BasiliKitap::yazdir() const {
    std::cout << "Basili Kitap: \"" << baslik_ << "\" - " << yazar_.tamAd()
              << " (" << yil_ << "), " << sayfaSayisi_ << " sayfa\n";
}
```

`kalitim_demo.cpp`'yi `#include "materyal.hpp"` ve `#include <iostream>` ile başlatın; `int main()` içine `using namespace kitaplik;` yazıp aşağıdaki bloğu ekleyin:

```
    BasiliKitap bk("Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320);
    std::cout << "Devralinan baslik(): " << bk.baslik() << '\n';
    bk.yazdir();
```

Terminal

```
$ g++ -std=c++20 kalitim_demo.cpp materyal.cpp -o kalitim_demo
$ ./kalitim_demo
```

Beklenen sonuç: Program derlenir ve çalışır. Önce kurucuların izleme satırlarını (`taban önce`) görürsünüz; sonra devralınan `baslik()` erişimcisinin `Etkili C++` döndürdüğünü ve `bk.yazdir()`'in türetilmiş sürümü (ortak bilgi *ve* sayfa sayısı) bastığını; program biterken de yıkıcıların *ters* sırayla çalıştığını gözlersiniz.

Beklenen Çıktı

```
Materyal kuruldu
BasiliKitap kuruldu
Devralinan baslik(): Etkili C++
Basili Kitap: "Etkili C++" - Scott Meyers (ABD) (2005), 320 sayfa
BasiliKitap yikildi
Materyal yikildi
```

Analoji

Kalıtım, "taşıt" kategorisine benzer: otomobil, kamyon, motosiklet hepsi *bir* taşıttır (ortak: tekerlek, motor, gaz pedalı), ama her biri kendine özgü de davranır. Ortak kısmı "taşıt" tabanında bir kez tanımlar, alt türlerde tekrarlamazsınız — tıpkı `baslik_`/`yazar_`/`yil_`'i `Materyal`'de bir kez yazıp `BasiliKitap`'a devrettiğiniz gibi.

Kontrol Noktası — devam etmeden doğrulayın

`materyal.hpp`'de `Yazar`, somut `Materyal` (protected alanlar + `virtual yazdir` + `virtual` yıkıcı) ve `BasiliKitap : public Materyal` bildirimlerinin bulunduğundan; `materyal.cpp`'de dört gövdenin (Materyal kurucu/yazdir/yıkıcı ve BasiliKitap kurucu/yazdir/yıkıcı) izleme satırlarıyla yazıldığından emin olun. `./kalitim_demo` çıktısının ilk satırı `Materyal kuruldu`, ikincisi `BasiliKitap kuruldu` ise kurucu zinciri doğru kurulmuş demektir.

2 Kurucu Zinciri, Yıkıcı Sırası ve Dilimleme Tuzağı

(bkz. Ders notu Bölüm 6, Ünite 2 — Kurucu Zinciri, Yıkıcı Sırası, Nesne Dilimlemesi (Slicing) Tuzağı.)

Adım 3: Kurucu ve yıkıcı çağrı sırasını deneyle gözlemleyin

Ne yapacağız? `kalitim_demo.cpp`'nin `main`'ine, önceki bloğun altına aşağıdaki bloğu ekleyip programı yeniden derleyip çalıştıracaksınız. Blok, bir iç kapsam (`{ ... }`) içinde geçici bir `BasiliKitap` oluşturur ve kapsam bitince onu yıktırır.

Neden? İzleme satırları sayesinde sırayı *deneyle* görürsünüz: bir nesne kurulurken **önce taban, sonra türetilmiş** (`Materyal kuruldu` → `BasiliKitap kuruldu`); yıkılırken **tersten — önce türetilmiş, sonra taban** (`BasiliKitap yikildi` → `Materyal yikildi`). Bu sıra rastlantı değildir: türetilmişin kurucusu, gövdesi çalışmadan önce tabanın tam kurulmuş olmasına güvenir; yıkımda da türetilmiş kendi kaynağını tabandan önce bırakmalıdır.

```
18    std::cout << "-- kapsam basliyor --\n";
19    {
20       BasiliKitap gecici("Temiz Kod", Yazar("Robert C. Martin", "ABD"), 2008,
21                          464);
22    }
23    std::cout << "-- kapsam bitti --\n";
```

Terminal

```
$ g++ -std=c++20 kalitim_demo.cpp materyal.cpp -o kalitim_demo
$ ./kalitim_demo
```

Beklenen sonuç: Çıktı, Adım 2'deki dört satırın ardından iç kapsamın izini de basar: `-- kapsam basliyor --`, sonra `Materyal kuruldu` → `BasiliKitap kuruldu` (kurulum sırası), sonra `BasiliKitap yikildi` → `Materyal yikildi` (yıkım tersi), sonra `-- kapsam bitti --`. En sonda `bk` yıkılırken yine türetilmiş-önce/taban-sonra sırasını görürsünüz.

Beklenen Çıktı

```
Materyal kuruldu
BasiliKitap kuruldu
Devralinan baslik(): Etkili C++
Basili Kitap: "Etkili C++" - Scott Meyers (ABD) (2005), 320 sayfa

-- kapsam basliyor --
Materyal kuruldu
BasiliKitap kuruldu
BasiliKitap yikildi
Materyal yikildi
-- kapsam bitti --
BasiliKitap yikildi
Materyal yikildi
```

Analoji

Kurucu zinciri ve yıkım sırası bina inşasına benzer: önce temel (taban), sonra üst katlar (türetilmiş) kurulur; yıkım tersinedir — önce üst katlar, en son temel. Temeli atmadan üst katı öremez, üst katları sökmeden temeli boşaltamazsınız.

Adım 4: Nesne dilimlemesi (slicing) tuzağını gözleyin

Ne yapacağız? `main`'e, önceki bloğun altına aşağıdaki bloğu ekleyeceksiniz. Blok bir `BasiliKitap`'ı bir `Materyal` değişkenine *değerle* kopyalar (`Materyal m = bk;`) ve hem `m.yazdir()`'i hem `bk.yazdir()`'i çağırarak ikisini karşılaştırır.

Neden? `Materyal m = bk;` yazınca `Materyal`'in kopya kurucusu yalnız **taban alt-nesnesini** kopyalar; `sayfaSayisi_` ve türetilmişe özgü davranış **kesilir** — buna **nesne dilimlemesi** denir. Sonuçta `m` artık "salt bir `Materyal`"tır ve `m.yazdir()` *taban* sürümünü çağırır (çıktıda sayfa sayısı *yoktur*); oysa `bk.yazdir()` türetilmiş sürümü basar. Bu, değerle kopyalamanın sinsi bir yan etkisidir ve ileride materyalleri bir kapta neden *işaretçiyle* tutacağımızın (Ünite 4) gerekçesidir.

```
29    Materyal m = bk; // slicing: yalniz Materyal alt-nesnesi kopyalanir
30    std::cout << "m.yazdir() (dilimlenmis, taban surumu) : ";
31    m.yazdir();
32    std::cout << "bk.yazdir() (turetilmis surumu) : ";
33    bk.yazdir();
```

Terminal

```
$ g++ -std=c++20 kalitim_demo.cpp materyal.cpp -o kalitim_demo
$ ./kalitim_demo
```

Beklenen sonuç: Çıktının sonuna, Adım 3'ün izinden sonra iki karşılaştırma satırı eklenir. `m.yazdir()` *taban* sürümünü basar — `Materyal: "Etkili C++" - Scott Meyers (ABD) (2005)` — **sayfa sayısı yoktur**; `bk.yazdir()` ise `Basili Kitap: ... , 320 sayfa` basar. İki satırın farkı slicing'i gösterir. (Uzun satırlar terminaliniz darsa alttaki gibi iki fiziksel satıra sarabilir; içerik aynıdır.)

Beklenen Çıktı

```
m.yazdir()  (dilimlenmis, taban surumu) : Materyal: "Etkili C++" - Scott
Meyers (ABD) (2005)
bk.yazdir() (turetilmis surumu)          : Basili Kitap: "Etkili C++" - Scott
Meyers (ABD) (2005), 320 sayfa
```

Ortak Hata / Tuzak

Dilimleme derleme hatası *vermez* — program sessizce "yarısı kayıp" bir nesneyle çalışmaya devam eder. Bu yüzden bir taban türünü **değerle** (kopya/atama) alan her yerde dikkatli olun. Çokbiçimliliği korumak istiyorsanız nesneyi değerle değil, **taban işaretçisi/referansı** üzerinden tutun — bunu Ünite 3 ve 4'te yapacaksınız.

Kontrol Noktası — devam etmeden doğrulayın

`./kalitim_demo` çıktınızda üç şeyi doğrulayın: (1) kurulum sırası `Materyal` → `BasiliKitap`; (2) yıkım sırası `BasiliKitap` → `Materyal`; (3) `m.yazdir()` satırında *sayfa sayısı yok*, `bk.yazdir()` satırında *var*. Üçü de tutuyorsa kurucu zincirini, yıkıcı sırasını ve slicing'i doğru gözlemlemişsiniz demektir.

3 `virtual` Gönderim, Sanal Yıkıcı ve Soyut Sınıf

(bkz. Ders notu Bölüm 6, Ünite 3 — `virtual` Fonksiyonlar, Çalışma-Zamanı Çokbiçimlilik, `override`, Sanal Yıkıcı, Saf Sanal + Soyut Sınıf.)

Adım 5: `EKitap` ile `SesliKitap`'ı türetip sanal gönderimi görün

Ne yapacağız? `materyal.hpp`'ye iki türetilmiş sınıf daha ekleyeceksiniz: `EKitap` (ek alan `boyutMB_`) ve `SesliKitap` (ek alanlar `sureDk_`, `seslendiren_`); ikisi de `public Materyal`'den türer, `yazdir()`'i `override` eder ve bir `override` yıkıcıya sahiptir. `materyal.cpp`'ye kurucu ve `yazdir` gövdelerini yazın (yıkıcı gövdelerini `BasiliKitap`'taki kalıba göre kendiniz ekleyin). Sonra `polimorfizm_demo.cpp` adlı yeni bir `main` dosyası oluşturup ilk bloğu yazın.

Neden? Artık üç kardeş türünüz var. `Materyal& r = bk;` satırında `r`'nin *statik* tipi `Materyal`, *dinamik* türü `BasiliKitap`'tır. `yazdir()` `virtual` olduğu için `r.yazdir()` çağrısı **çalışma zamanında** gerçek türe (`BasiliKitap::yazdir`) çözülür — buna **dinamik gönderim** denir. `virtual` olmasaydı çağrı statik tipe (`Materyal::yazdir`) bağlanırdı. Aynı ilke bir `Materyal*` dizisinde de işler: tek bir döngüde her eleman *kendi türüne* göre yazdırılır; farklı türler tek arayüz altında farklı davranır (çokbiçimlilik).

```
class EKitap : public Materyal {
public:
   EKitap(std::string baslik, Yazar yazar, int yil, double boyutMB);
   void yazdir() const override;
   ~EKitap() override;

private:
   double boyutMB_;
};
```

```
class SesliKitap : public Materyal {
public:
   SesliKitap(std::string baslik, Yazar yazar, int yil, int sureDk,
           std::string seslendiren);
   void yazdir() const override;
   ~SesliKitap() override;

private:
   int sureDk_;
   std::string seslendiren_;
};
```

```
EKitap::EKitap(std::string baslik, Yazar yazar, int yil, double boyutMB)
   : Materyal(std::move(baslik), std::move(yazar), yil), boyutMB_(boyutMB) {
   std::cout << "EKitap kuruldu\n";
}
```

```
void EKitap::yazdir() const {
   std::cout << "E-Kitap: \"" << baslik_ << "\" - " << yazar_.tamAd()
           << " (" << yil_ << "), " << boyutMB_ << " MB\n";
}
```

```
SesliKitap::SesliKitap(std::string baslik, Yazar yazar, int yil, int sureDk,
                 std::string seslendiren)
   : Materyal(std::move(baslik), std::move(yazar), yil), sureDk_(sureDk),
     seslendiren_(std::move(seslendiren)) {
   std::cout << "SesliKitap kuruldu\n";
}
```

Adım 5 (devamı)

```
void SesliKitap::yazdir() const {
   std::cout << "Sesli Kitap: \"" << baslik_ << "\" - " << yazar_.tamAd()
           << " (" << yil_ << "), " << sureDk_ << " dk, seslendiren: "
           << seslendiren_ << '\n';
}
```

`polimorfizm_demo.cpp`'yi `#include "materyal.hpp"`, `#include <iostream>` ve `#include <memory>` ile başlatın; `main` başında `using namespace kitaplik;` yazıp aşağıdaki bloğu ekleyin:

```
   BasiliKitap bk("Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320);
   EKitap ek("C++ Temelleri", Yazar("Ayse Yilmaz", "TR"), 2020, 4.5);
   SesliKitap sk("Kisa Hikayeler", Yazar("Mehmet Demir", "TR"), 2019, 180,
             "Ali Kaya");

   Materyal& r = bk;
   std::cout << "Materyal& uzerinden (statik tip Materyal, dinamik tur "
             "BasiliKitap):\n";
   r.yazdir();

   std::cout << "\nMateryal* dizisi uzerinden cokbicimli yazdirma:\n";
   Materyal* dizi[] = {&bk, &ek, &sk};
   for (Materyal* p : dizi) {
      p->yazdir();
   }
```

Terminal

```
$ g++ -std=c++20 polimorfizm_demo.cpp materyal.cpp -o polimorfizm_demo
$ ./polimorfizm_demo
```

Beklenen sonuç: Üç materyal kurulduktan sonra, hem `Materyal&` referansı hem de `Materyal*` dizisi üzerinden yapılan `yazdir()` çağrıları **türetilmiş** sürümleri basar: basılı, e-kitap ve sesli satırları her biri kendi ek alanıyla çıkar. Statik tip `Materyal` olsa da dinamik tür belirleyicidir. Program biterken üç nesne ters sırayla yıkılır. (Uzun sesli kitap satırı dar terminalde iki fiziksel satıra sarabilir; içerik aynıdır.)

Adım 5 (devamı)

Beklenen Çıktı

```
Materyal kuruldu
BasiliKitap kuruldu
Materyal kuruldu
EKitap kuruldu
Materyal kuruldu
SesliKitap kuruldu
Materyal& uzerinden (statik tip Materyal, dinamik tur BasiliKitap):
Basili Kitap: "Etkili C++" - Scott Meyers (ABD) (2005), 320 sayfa

Materyal* dizisi uzerinden cokbicimli yazdirma:
Basili Kitap: "Etkili C++" - Scott Meyers (ABD) (2005), 320 sayfa
E-Kitap: "C++ Temelleri" - Ayse Yilmaz (TR) (2020), 4.5 MB
Sesli Kitap: "Kisa Hikayeler" - Mehmet Demir (TR) (2019), 180 dk,
seslendiren: Ali Kaya
SesliKitap yikildi
Materyal yikildi
EKitap yikildi
Materyal yikildi
BasiliKitap yikildi
Materyal yikildi
```

Analoji

Sanal gönderim, kumandadaki "aç" tuşuna benzer: aynı tuş (arayüz: `yazdir()`) televizyonda ekranı, klimada soğutmayı başlatır — davranış, karşıdaki gerçek cihaza (nesnenin gerçek türüne) göre değişir.

Adım 6: Sanal yıkıcının taban işaretçiyle doğru temizlik yapmasını doğrulayın

Ne yapacağız? `polimorfizm_demo.cpp`'nin `main`'ine, önceki bloğun altına aşağıdaki bloğu ekleyeceksiniz. Blok bir `BasiliKitap`'ı bir `std::unique_ptr<Materyal>` ile (taban işaretçisi) tutar ve `p.reset()` ile yıktırır.

Neden? Bir `Materyal` işaretçisi üzerinden silme yapıldığında, yalnız tabanın değil türetilmişin de yıkıcısının çalışması gerekir. Bunu sağlayan şey, `~Materyal()`'in `virtual` olmasıdır: sanal yıkıcı sayesinde `p.reset()` önce `BasiliKitap` yıkıcısını, sonra `Materyal` yıkıcısını çalıştırır. `~Materyal()` `virtual` *olmasaydı* yalnız taban yıkıcısı koşar, türetilmişin ek kaynağı temizlenmez — tanımsız davranış oluşurdu. Taban işaretçiyle silinebilecek her polimorfik tabanda yıkıcı `virtual` olmalıdır.

```
32    std::cout << "-- unique_ptr<Materyal> olusturuluyor --\n";
33    std::unique_ptr<Materyal> p = std::make_unique<BasiliKitap>(
34       "Tasarim Desenleri", Yazar("Erich Gamma", "ABD"), 1994, 395);
35    std::cout << "-- unique_ptr sifirlaniyor (yikim tetiklenir) --\n";
36    p.reset();
37    std::cout << "-- yikim tamamlandi --\n";
```

Terminal

```
$ g++ -std=c++20 polimorfizm_demo.cpp materyal.cpp -o polimorfizm_demo
$ ./polimorfizm_demo
```

Adım 6 (devamı)

Beklenen sonuç: Önceki çıktının ardından, bu bloğun izi görünür: `unique_ptr` oluşturulunca kurucular (`Materyal kuruldu` → `BasiliKitap kuruldu`); `p.reset()` çağrılınca yıkıcılar *türetilmiş-önce/taban-sonra* sırayla (`BasiliKitap yikildi` → `Materyal yikildi`) çalışır. İki yıkıcı satırının da görünmesi, sanal yıkıcının doğru kurulduğunu kanıtlar.

Beklenen Çıktı

```
-- unique_ptr<Materyal> olusturuluyor --
Materyal kuruldu
BasiliKitap kuruldu
-- unique_ptr sifirlaniyor (yikim tetiklenir) --
BasiliKitap yikildi
Materyal yikildi
-- yikim tamamlandi --
```

Ortak Hata / Tuzak

Polimorfik bir tabanın yıkıcısını `virtual` yapmayı unutmak en sessiz hatalardan biridir: kod derlenir, çoğu zaman "çalışır gibi" görünür, ama taban işaretçiyle silinen türetilmiş nesnelerin kaynağı sızar. Kuralı ezberleyin: bir sınıftan türetme yapılacaksa ve taban işaretçiyle silinecekse, yıkıcı `virtual` olmalıdır.

Adım 7: Saf sanal fonksiyon ve soyut sınıf sözdizimini `Sekil` ile deneyin

Ne yapacağız? `sekil.hpp` adlı yeni bir dosya oluşturup içine soyut bir `Sekil` sınıfı (`virtual double alan() const = 0;` ve `virtual ~Sekil() = default;`) ile ondan türeyen `Daire` ve `Kare` sınıflarını (`alan()`'ı `override` ederler) yazacaksınız. Sonra `polimorfizm_demo.cpp`'ye `#include "sekil.hpp"` ekleyip `main`'e aşağıdaki bloğu koyun.

Neden? `= 0` bir fonksiyonu **saf sanal** yapar: taban onu *tanımlamaz*, her türetilmiş tanımlamak **zorundadır**. En az bir saf sanalı olan sınıf **soyut**tur ve doğrudan örneklenemez — `Sekil s;` derleme hatası verir (blokta yorumla belirtilmiştir). Yalnız türetilmişleri (`Daire`, `Kare`) örneklenir ve `Sekil*` üzerinden `alan()` çokbiçimli çağrılır. Bu, bir tipe "şu işlevi mutlaka uygula" sözleşmesi dayatmanın yoludur.

```
7  class Sekil {
8  public:
9     virtual double alan() const = 0;
10    virtual ~Sekil() = default;
11 };

15 class Daire : public Sekil {
16 public:
17    explicit Daire(double yaricap) : yaricap_(yaricap) {}
18    double alan() const override { return 3.14159265358979 * yaricap_ * yaricap_; }
19
20 private:
21    double yaricap_;
22 };
23
24 class Kare : public Sekil {
25 public:
26    explicit Kare(double kenar) : kenar_(kenar) {}
27    double alan() const override { return kenar_ * kenar_; }
```

Adım 7 (devamı)

```
28
29 private:
30    double kenar_;
31 };
```

```
43    // Sekil gov; // DERLEME HATASI: soyut sinif (saf sanal alan()) dogrudan
44    // // orneklenemez.
45    Daire d(3.0);
46    Kare kr(4.0);
47    Sekil* sekiller[] = {&d, &kr};
48    for (Sekil* s : sekiller) {
49       std::cout << "alan = " << s->alan() << '\n';
50    }
```

Terminal

```
$ g++ -std=c++20 polimorfizm_demo.cpp materyal.cpp -o polimorfizm_demo
$ ./polimorfizm_demo
```

Beklenen sonuç: Çıktının en sonunda (önceki bloklardan sonra) iki alan değeri basılır: `Daire(3.0)` için `alan = 28.2743`, `Kare(4.0)` için `alan = 16`. Bunlar `Sekil*` üzerinden çokbiçimli hesaplanır. `Sekil gov;` satırının başındaki yorumu kaldırırsanız program *derlenmez* ("cannot declare variable ... to be of abstract type") — soyut sınıf doğrudan örneklenemez.

Beklenen Çıktı

```
alan = 28.2743
alan = 16
```

Dikkat

`Materyal`'i de `yazdir()`'i saf sanal (`= 0`) yaparak soyutlaştırabilirdiniz; bu, Ünite 2'deki slicing tuzağını tümden imkânsız kılardı (değerle örneklenemeyen bir taban dilimlenemez). Bu hafta `Materyal` bilinçli olarak **somut** kalıyor ki slicing'i gözleyebilesiniz; tam soyutlaştırma Hafta 7'nin (Çokbiçimlilik) konusudur.

Kontrol Noktası — devam etmeden doğrulayın

`./polimorfizm_demo` çıktınızda: (1) `Materyal&` ve `Materyal*` üzerinden çağrıların *türetilmiş* yazdır satırlarını bastığını; (2) `p.reset()` sonrası hem `BasiliKitap yikildi` hem `Materyal yikildi` satırlarının çıktığını; (3) `alan = 28.2743` ve `alan = 16` satırlarının göründüğünü doğrulayın. Üçü de tutuyorsa sanal gönderimi, sanal yıkıcıyı ve soyut sınıfı doğru kurmuşsunuz demektir.

4 Kompozisyon ve Çokbiçimli `Kutuphane`

(bkz. Ders notu Bölüm 6, Ünite 4 — Kompozisyon (has-a) vs Kalıtım (is-a): Ne Zaman Hangisi? Çokbiçimli `Kutuphane`.)

Adım 8: `Kutuphane`'yi kompozisyonla kurup çokbiçimli yazdırın

Ne yapacağız? `materyal.hpp`'ye bir `Kutuphane` sınıfı ekleyeceksiniz: `private` üyesi `std::vector<std::unique_ptr<Materyal>> raf_;` `public` arayüzü `ekle(std::unique_ptr<Materyal> m)` ve `hepsiniYazdir() const`. `materyal.cpp`'ye ikisinin gövdesini yazın (`ekle` `raf_.push_back(std::move(m))` yapar, `hepsiniYazdir` `raf_` üzerinde tek döngüyle her `m->yazdir()`'i çağırır). Sonra `kutuphane_demo.cpp` adlı yeni bir `main` dosyası oluşturup üç türü tek kaba koyun.

Neden? `Kutuphane` bir `Materyal` *değildir*; materyallere *sahiptir* — bu bir **has-a** (kompozisyon) ilişkisidir, kalıtım değil. Bu yüzden `Kutuphane`'yi `std::vector`'dan *türetmezsiniz* (o bir vektör değildir); bir vektörü *üye* olarak tutar. Kap neden `std::vector<std::unique_ptr<Materyal>>`? İki gerekçe: (1) materyalleri *değerle* (`std::vector<Materyal>`) tutmak onları **dilimler** ve dinamik türü kaybederdi (Ünite 2); işaretçi gerçek türü korur, böylece `m->yazdir()` çokbiçimli çalışır. (2) `unique_ptr` sahipliği RAII ile yönetir (Hafta 4) — `Kutuphane` yıkılınca tüm materyaller otomatik silinir.

```
// has-a: Kutuphane bir Materyal DEGILDIR, materyallere sahiptir (kompozisyon).
class Kutuphane {
public:
   void ekle(std::unique_ptr<Materyal> m);
   void hepsiniYazdir() const;

private:
   std::vector<std::unique_ptr<Materyal>> raf_;
};
```

```
void Kutuphane::ekle(std::unique_ptr<Materyal> m) {
   raf_.push_back(std::move(m));
}
```

```
void Kutuphane::hepsiniYazdir() const {
   for (const auto& m : raf_) {
      m->yazdir();
   }
}
```

`kutuphane_demo.cpp`'yi `#include "materyal.hpp"`, `#include <iostream>` ve `#include <memory>` ile başlatın; `main` içinde `using namespace kitaplik;` yazıp aşağıdaki bloğu ekleyin:

```
   Kutuphane k;
   k.ekle(std::make_unique<BasiliKitap>(
      "Etkili C++", Yazar("Scott Meyers", "ABD"), 2005, 320));
   k.ekle(std::make_unique<EKitap>("C++ Temelleri",
                           Yazar("Ayse Yilmaz", "TR"), 2020, 4.5));
   k.ekle(std::make_unique<SesliKitap>("Kisa Hikayeler",
                              Yazar("Mehmet Demir", "TR"), 2019, 180,
                              "Ali Kaya"));

   k.hepsiniYazdir();
```

```
>_ Terminal

$ g++ -std=c++20 kutuphane_demo.cpp materyal.cpp -o kutuphane_demo
$ ./kutuphane_demo
```

Adım 8 (devamı)

Beklenen sonuç: Üç materyal kurulduktan sonra `hepsiniYazdir()`, tek döngüde her materyali **kendi türüne göre** basar: basılı satır sayfa sayısıyla, e-kitap satırı MB'la, sesli satır süre ve seslendirenle. Farklı türler tek kapta, tek döngüde doğru davranır — işte çokbiçimlilik. `Kutuphane` kapsam dışına çıkınca üç materyal ters sırayla yıkılır. (Uzun sesli kitap satırı dar terminalde iki fiziksel satıra sarabilir; içerik aynıdır.)

Beklenen Çıktı

```
Materyal kuruldu
BasiliKitap kuruldu
Materyal kuruldu
EKitap kuruldu
Materyal kuruldu
SesliKitap kuruldu
Basili Kitap: "Etkili C++" - Scott Meyers (ABD) (2005), 320 sayfa
E-Kitap: "C++ Temelleri" - Ayse Yilmaz (TR) (2020), 4.5 MB
Sesli Kitap: "Kisa Hikayeler" - Mehmet Demir (TR) (2019), 180 dk,
seslendiren: Ali Kaya
SesliKitap yikildi
Materyal yikildi
EKitap yikildi
Materyal yikildi
BasiliKitap yikildi
Materyal yikildi
```

Analoji

Çokbiçimli koleksiyon bir meyve sepetine benzer: sepet elma, muz, portakal tutar; hepsi *bir* meyvedir. Sepeti tek tek gezip her birini "tartarsınız" — aynı işlem (`yazdir()`) her meyvede kendi türüne göre sonuç verir.

Adım 9: Kendiniz ekleyin: dördüncü bir materyal türü türetin

Ne yapacağız? Şimdiye kadar öğrendiğiniz kalıbı kullanarak `Materyal`'den *yeni* bir türetilmiş sınıf oluşturup `Kutuphane`'ye ekleyeceksiniz. Aşağıdakilerden **birini** seçin (ya da benzerini tasarlayın).

Neden? Bir sınıfı sıfırdan türetmek, bu haftanın tüm kararlarını kendi başınıza vermenizi sağlar: `public` kalıtım, kurucu zinciri, `override` edilmiş `yazdir()` ve `Kutuphane`'ye `unique_ptr` olarak ekleme. Doğru kurulursa yeni türünüz de `hepsiniYazdir()` döngüsünde kendi türüne göre otomatik basılır — çokbiçimliliğin gücü budur: mevcut döngüye hiç dokunmadan yeni bir tür eklersiniz.

- **`DergiSayisi`**: ek alanlar `int sayi_` (sayı numarası) ve `std::string ay_`; `yazdir()`'i ortak bilgiyi + `"Sayi N, ay"` bilgisini basacak biçimde `override` edin. Kurucu zincirinde önce `Materyal(...)`'i çağırmayı unutmayın.
- **`Tez`**: ek alanlar `std::string universite_` ve `int sayfaSayisi_`; `yazdir()`'i tez bilgisini basacak biçimde `override` edin.

Yeni sınıfı `kutuphane_demo.cpp`'de `k.ekle(std::make_unique<DergiSayisi>(...));` gibi bir satırla kaba ekleyip programı yeniden çalıştırın.

Adım 9 (devamı)

Beklenen sonuç: Yeni türünüz kaba eklenince `hepsiniYazdir()` çıktısına — mevcut döngüyü hiç değiştirmeden — kendi türüne özgü bir satır daha eklenir. Kuruluşta `Materyal kuruldu` → `<YeniTur> kuruldu`, yıkımda ters sıra izini de görürsünüz.

İpucu

Yeni türetilmiş sınıf yazarken üç şeyi mekanik olarak kopyalayın: (1) başlıkta `: public Materyal`; (2) kurucuda başlatma listesinin *başında* `Materyal(std::move(baslik), std::move(yazar), yil)`; (3) `yazdir()` bildiriminin sonunda `override`. Tabanın yıkıcısı zaten `virtual` olduğu için yeni türünüz `Kutuphane`'de sorunsuz yıkılır.

Ortak Hata / Tuzak

`yazdir()`'in imzasını tabanla *birebir* eşleştirin (`void yazdir() const`). `const`'u unutur ya da imzayı değiştirirseniz, `override` bir *derleme hatası* verir ("does not override") — ki bu iyidir: sessizce yeni bir fonksiyon oluşturup çokbiçimli çağrının tabana düşmesini engeller. `override` yazmasaydınız bu hatayı almaz, sinsi bir biçimde yanlış sürüm çağrılırdı.

Sorun Giderme

Belirti	Olası neden	Çözüm
`no matching function ... Materyal::Materyal()` veya `no default constructor`	Türetilmiş kurucuda başlatma listesinde taban `Materyal(...)` çağrılmadı	Türetilmiş kurucunun başlatma listesine `: Materyal(baslik, yazar, yil)`'i ekleyin (kurucu zinciri)
`'baslik_' is a protected member` / erişilemiyor (sınıf dışından)	`protected` alanlara sınıf hiyerarşisi *dışından* erişmeye çalıştınız	`protected` yalnız türetilmişlere açıktır; dışarıdan `baslik()` gibi `public` erişimciyi kullanın
`m.yazdir()` taban sürümünü basıyor (sayfa sayısı yok) ama siz türetilmiş beklediniz	Nesneyi *değerle* bir `Materyal`'e kopyaladınız (slicing)	Çokbiçimliliği korumak için nesneyi `Materyal&` ya da `Materyal*`/`unique_ptr<Materyal>` üzerinden tutun
`Materyal*` üzerinden çağrı hep taban sürümünü basıyor	`yazdir()` tabanda `virtual` bildirilmedi	Taban `yazdir()`'i `virtual`, türetilmişleri `override` yapın
Türetilmiş nesnenin ek kaynağı temizlenmiyor / sızıyor	`~Materyal()` `virtual` değil	Taban yıkıcısını `virtual` yapın (polimorfik tabanlarda zorunlu)
`cannot declare variable ... abstract type 'Sekil'`	Soyut bir sınıfı doğrudan örneklemeye çalıştınız	Soyut sınıf örneklenemez; yalnız türetilmişini (`Daire`/`Kare`) oluşturun
`'yazdir' marked 'override' but does not override`	Türetilmişin imzası tabanla eşleşmiyor (çoğu zaman eksik `const`)	İmzayı tabanla birebir aynı yapın: `void yazdir() const override;`

5 • İzleme Aktarım — Başka Nerede?

Bu föyde kurduğunuz kalıtım + kompozisyon + çokbiçimlilik iskeleti tek bir projeye özgü değildir; her yerde yeniden karşınıza çıkar:

- **Kitaplık projesine:** Oluşturduğunuz `Materyal` hiyerarşisi ve `Kutuphane` kabı, Hafta 7'de (Çokbiçimlilik) doğrudan büyütülecek: `Materyal`'i *soyut* yapıp sanal bir `ozet()` ekleyerek bir `Materyal*` koleksiyonunu tam anlamıyla çokbiçimli işleyeceksiniz. Bugünkü somut iskelet, o haftanın başlangıç noktasıdır.
- **Karar kalıbına:** "…dır" testini geçen her ilişki bir kalıtım (is-a), "…sahiptir/…içerir" testini geçen her ilişki bir kompozisyon (has-a) adayıdır. Belirsizse kompozisyonu tercih edin (daha gevşek bağ, daha esnek tasarım). Bu karar, Hafta 13'te istisna türlerinde (`KitapBulunamadi` *bir* `std::exception`'dır) ve YG II'de Qt arayüz sınıflarında (bir düğme *bir* `QWidget`'tır) aynen tekrar eder.
- **Çokbiçimli koleksiyona:** Farklı türleri tek arayüz altında tek döngüde işleme örüntüsü, bir çizim programında farklı şekilleri, bir oyun döngüsünde farklı nesneleri işlerken de aynıdır. `Kutuphane`'nin kabı ise Hafta 9'da generic bir `Liste<T>`'ye, Hafta 11'de bir `AramaAgaci`'na evrilecek — ama "kütüphane koleksiyona sahiptir" (has-a) ilişkisi değişmeden kalacak.